

PROMISE

INTERVIEW QUESTION-139



Follow on



@Duvvuru Kishore



In asynchronous JavaScript, **promises** are crucial for handling tasks that take time to complete, like server requests.

What is a Promise?

A promise is an object that represents the **eventual completion or failure of an asynchronous operation** and its resulting value. It allows you to write cleaner, more manageable code by handling asynchronous tasks efficiently.



A diagram consisting of a black rectangular background. On the left side, there is a green circle with the word 'fulfilled' written inside in a green, monospace-style font. On the right side, there is a red circle with the word 'rejected' written inside in a red, monospace-style font.

fulfilled

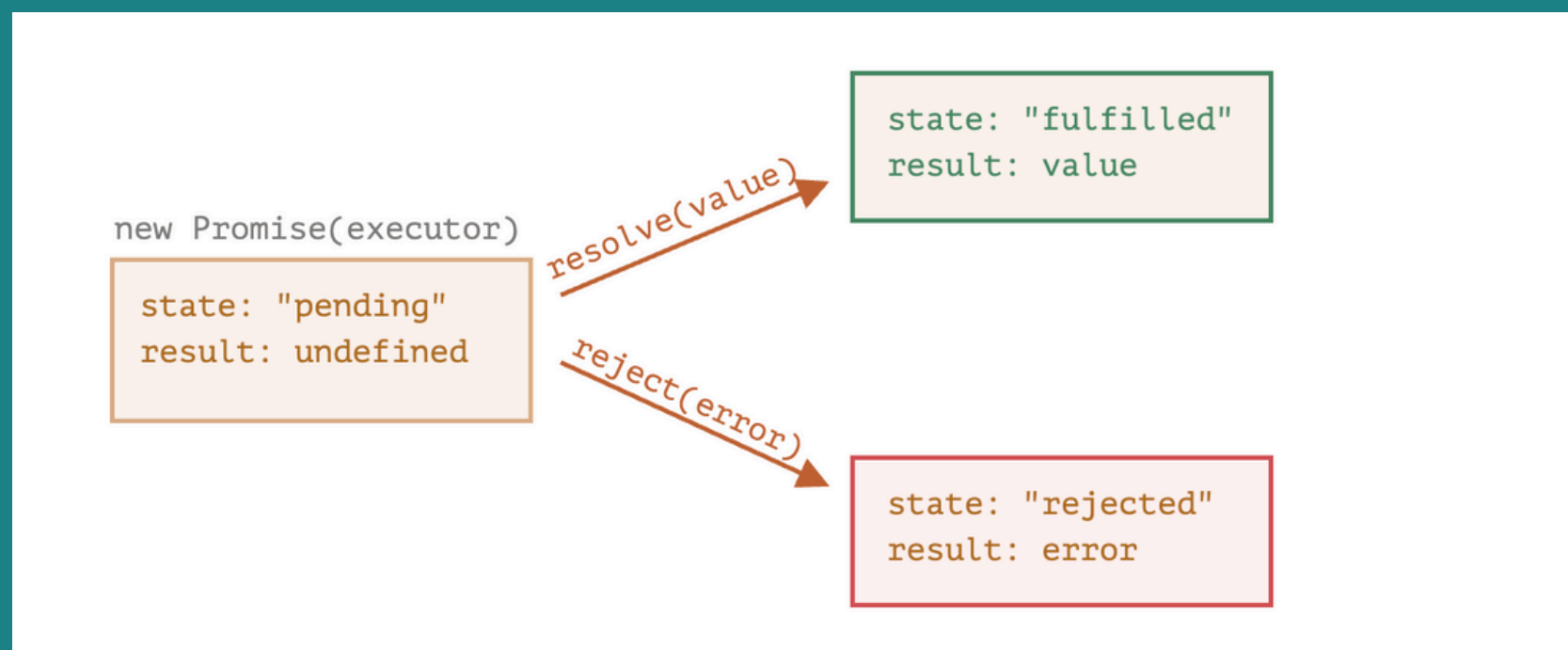
rejected

Promise States

Pending: The initial state. The operation hasn't completed yet.

Fulfilled: The operation completed successfully, and you have a result.

Rejected: The operation failed, and you have a reason for the failure.



Using Promises

.then(): Once a **promise is fulfilled**, you can use **.then()** to specify what should happen next. It's like saying, "When this task is done, do this next step."

.catch(): If a **promise is rejected**, **.catch()** allows you to handle the error, ensuring your application can gracefully recover or inform the user.

Promise Example

```
// Fetch user data and handle the promise in one function
function fetchAndDisplayUserData(userId) {
  fetch(`https://jsonplaceholder.typicode.com/users/${userId}`)
    .then((response) => {
      if (!response.ok) {
        throw new Error('Network response was not ok');
      }
      return response.json(); // Parse the JSON data from the response
    })
    .then((userData) => {
      console.log('User Data:', userData); // Output user data
    })
    .catch((error) => {
      console.error('There was a problem with the fetch operation:', error);
    });
}
```

Engaging Analogy: The Pizza Order

Imagine promises as placing an order for a delicious pizza:

Pending: You place the order and wait. The promise is pending.

Fulfilled: The pizza arrives hot and fresh. The promise is fulfilled, and you enjoy your meal.

Rejected: The pizzeria calls to say they're out of ingredients. The promise is rejected, and you need a backup plan.

By thinking of promises like this, you can better understand how they help manage asynchronous tasks in a logical, predictable way.

Drawbacks of Promises

While promises improve asynchronous handling, they can sometimes lead to complex and nested chains of `.then()` and `.catch()` calls, making code harder to read and maintain. This is often referred to as “**promise hell**”.

Promise hell example

```
fetchDataFromServer()
  .then((data) => {
    processData(data)
      .then((processedData) => {
        validateData(processedData)
          .then((isValid) => {
            if (isValid) {
              saveData(processedData)
                .then(() => {
                  console.log('Data saved successfully!');
                })
                .catch((error) => {
                  console.error('Error saving data:', error);
                });
            } else {
              console.error('Data is invalid');
            }
          })
          .catch((error) => {
            console.error('Error validating data:', error);
          });
        })
      .catch((error) => {
        console.error('Error processing data:', error);
      });
    })
  .catch((error) => {
    console.error('Error fetching data:', error);
  });
```

To address these drawback, ES2017 introduced **async/await**, allowing developers to write asynchronous code that looks and behaves like synchronous code:

async Function:

Declares a function that returns a promise.

await Keyword:

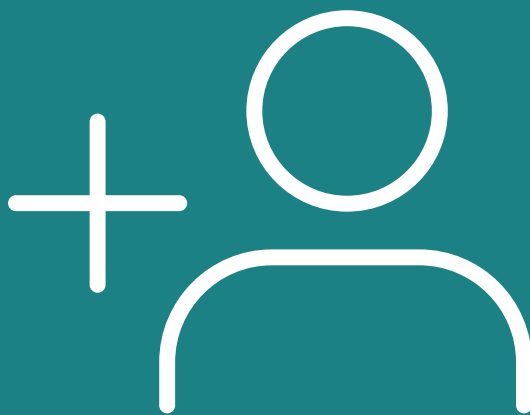
Pauses the execution of the async function, waiting for the promise to resolve or reject.

async/await example

```
async function fetchData() {  
  try {  
    let response = await fetch('https://api.example.com/data')  
    let data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Error fetching data:', error);  
  }  
}
```

Mastering **promises** and **async/await** is crucial for modern web development. They enable you to handle asynchronous operations effectively, improving code readability and maintainability.

Understanding these concepts will not only impress interviewers but also enhance your ability to build responsive, efficient applications.



Follow on 
@Duvvuru Kishore

